

Лабораторная работа 8

Основы объектно-ориентированного программирования

Введение

ООП — подход, при котором данные и действия над ними объединяются в классы. Класс описывает состояние (атрибуты) и поведение (методы). Объекты — конкретные экземпляры классов. Наследование позволяет переиспользовать код.

Цель работы: изучить синтаксис классов, создание объектов, инкапсуляцию и наследование.

Теоретическая часть

Класс определяется через `class Имя:`.

Метод `__init__` — конструктор, вызывается при создании объекта.

Первый параметр методов — `self`, ссылающийся на экземпляр.

Инкапсуляция реализуется соглашениями: `__protected`, `__private`.

Наследование: `class Child(Parent):`, вызов `super().__init__()`.

Задания

1. Создайте класс `Rectangle` с атрибутами `width` и `height`, методами `area()` и `perimeter()`. Создайте два объекта и выведите их площади и периметры.
2. Реализуйте класс `BankAccount` с приватным атрибутом `__balance`, методами `deposit(amount)`, `withdraw(amount)`, `get_balance()`. При попытке снять больше баланса выбрасывайте исключение.
3. Разработайте мини-проект «Библиотека»: класс `Book` (название, автор, статус), класс `Library` (список книг, методы выдачи, возврата, добавления, отображения с фильтром). Обеспечьте сохранение и загрузку в файл.
4. Создайте базовый класс `Shape` с методом `area()`, возвращающим 0. Создайте дочерние классы `Circle` и `Square`, переопределяющие `area()`. Продемонстрируйте полиморфизм: поместите несколько фигур в список и вычислите суммарную площадь.
5. Реализуйте класс `Stack` с методами `push(item)`, `pop()` и `is_empty()`. При попытке извлечь из пустого стека выбрасывайте собственное исключение `StackEmptyException`.
6. Добавьте в класс `Rectangle` метод `__str__`, возвращающий строку вида "Прямоугольник 3x4". Продемонстрируйте его работу через `print(объект)`.
7. Создайте класс `Point` с координатами `x`, `y`. Реализуйте метод `distance_to(other)`, вычисляющий евклидово расстояние между точками.
8. Напишите иерархию классов «Транспорт»: базовый `Vehicle` и наследники `Car`, `Bicycle`. Добавьте уникальные атрибуты и переопределите метод `description()`.
9. Создайте класс `Fraction` (числитель, знаменатель) с методами `__add__`, `__sub__`, `__mul__`, `__truediv__`. Упрощайте дробь после операций.
10. Реализуйте класс `Student` с полями `имя` и список оценок. Добавьте метод `average()`, а также статический метод `from_string(s)` для создания объекта из строки «Имя: оценки».
11. Напишите класс `Temperature` с приватным атрибутом `_celsius` и свойствами (`property`) для чтения/записи температуры в градусах Фаренгейта.

12. Создайте класс `Playlist` (плейлист песен) с методами добавления, удаления, перемешивания и сохранения/загрузки списка в файл.
13. Реализуйте класс `Polynomial`, позволяющий задавать многочлен списком коэффициентов и вычислять его значение для заданного x .
14. Используя наследование, создайте класс `Animal` и подклассы `Dog`, `Cat`.
Переопределите метод `speak()`, возвращающий соответствующее звукоподражание.
15. Напишите класс `Matrix` (двумерный список) с методами сложения, умножения на число и транспонирования.

Контрольные вопросы

1. Что такое класс и объект?
2. Зачем нужен параметр `self`?
3. Чем отличается наследование от композиции?
4. Как в Python обозначаются защищённые и приватные атрибуты?
5. Что такое полиморфизм?

Выводы

Освоены принципы ООП: инкапсуляция, наследование, полиморфизм. Созданы собственные классы, реализованы проекты с файловым сохранением.